Wstęp do Programowania*

Marcin Kubica

2022/2023

Outline

1. Wskaźnikowe struktury danych
   - Listy i kolejki
   - Drzewa binarne i BST
   - Drzewa Find-Union

Wskaźnikowe/dynamiczne struktury danych

Example

- Strukturę danych budujemy z wielu rekordów (struktur) połączonych odpowiednio wskaźnikami.
- Liczba i sposób połączenia rekordów zmienia się *dynamicznie*.
- Nowe rekordy tworzymy za pomocą `malloc`, a zbędne usuwamy za pomocą `free`.
- *Memory leak* — błąd polegający na gubieniu przydzielonej pamięci bez jej zwalniania.

Listy jednokierunkowe

Example

- Ciąg elementów — każdy w osobnej strukturze.
- W każdej strukturze mamy wskaźnik prowadzący do kolejnej.
- Lista = wskaźnik na pierwszy rekord (lub `NULL`).
- Przydatne np. do tworzenia stosów.

Listy jednokierunkowe

Example

```
typedef struct elem {
    int val;
    elem *next;
} elem;

typedef elem* lista;

lista push(lista l, int x) {
    lista nowy = (lista)malloc(sizeof(elem));
    if (nowy != NULL) {
        nowy->val = x;
        nowy->next = l;
    }
    return nowy;
}
```

Listy jednokierunkowe

Example

```
lista pop(lista l) {
    if (l == NULL) {
        return l;
    }
    lista n = l->next;
    free(l);
    return n;
}

lista reverse(lista l) {
    lista res = NULL;
    while(l) {
        res = push(res, l->val);
        l = pop(l);
    }
    return res;
```

Listy jednokierunkowe — wady i zalety

Example

- Operacje na początku listy $\Theta(1)$ (koszt faktyczny, nie zamortyzowany).
- Pozostałe operacje, raczej $O(n)$.
- Niewrażliwe na fragmentację pamięci.
- Narzut pamięciowy.

Listy dwukierunkowe

Example

- Wskaźniki łączą rekordy w obu kierunkach.
- Lista = wskaźnik na początek i koniec
- Pusta lista = $2 \times$ `NULL`.

Listy dwukierunkowe

Example

```
typedef struct elem {
    int val;
    elem *prev, *next;
} elem;

typedef struct lista {
    elem *head, *tail;
} lista;

lista create_list() {
    lista res = {
        .head = NULL,
        .tail = NULL
    };
    return res;
}
```

Listy dwukierunkowe

Example

```
bool isempty(const lista l) {
    return l.head == NULL;
}

int front(const lista l) {
    return l.head->val;
}

int back(const lista l) {
    return l.tail->val;
}
```

Listy dwukierunkowe

Example

```
void push_front(lista &l, int x) {
    elem *el = (elem*)malloc(sizeof(elem));
    el->val = x;
    el->next = l.head;
    el->prev = NULL;
    if (!isempty(l)) {
        l.head->prev = el;
    } else {
        l.tail = el;
    }
    l.head = el;
}

void push_back(lista &l, int x) {
analogicznie ...
```

Listy dwukierunkowe

Example

```
void pop_front(lista &l) {
    if (!isempty(l)) {
        elem *el = l.head;
        l.head = l.head->next;
        if (l.head) {
            l.head->prev = NULL;
        } else {
            l.tail = NULL;
        }
        free(el);
    }
}

void pop_back(lista &l) {
analogicznie ...
```

Listy dwukierunkowe

Example

- Faktyczny stały czas operacji na obu końcach.
- Przydatne jako kolejki FIFO/LIFO.
- Odporne na fragmentację pamięci.
- Narzut pamięciowy.

Dwustronne kolejki ze scalaniem i odwracaniem

Example

- Lista dwukierunkowa z dodatkowymi operacjami:
  - odwracania kolejki,
  - sklejania kolejek.
- Lista dwukierunkowa, bez określania kierunków dowiązań.
- Atrapy na końcach.
- Lista, to para dowiązań do atrap.

Dwustronne kolejki ze scalaniem i odwracaniem

Example

```
typedef struct elem {
    int val;
    elem *l1, *l2;
} elem;

typedef struct lista {
    elem *head, *tail;
} lista;
```

Dwustronne kolejki ze scalaniem i odwracaniem

Example

```
lista create_list() {
    lista res = {
        .head = (elem*)malloc(sizeof(elem)),
        .tail = (elem*)malloc(sizeof(elem))
    };
    // Co jak się nie uda?
    res.head->l1 = NULL;
    res.head->l2 = res.tail;
    res.tail->l1 = NULL;
    res.tail->l2 = res.head;
    return res;
}
```

Dwustronne kolejki ze scalaniem i odwracaniem

Example

```
bool isempty(const lista l) {
    return l.head->l2 == l.tail;
}
int front(const lista l) {
    return l.head->l2->val;
}
int back(const lista l) -- analogicznie

void link(elem *el, elem *old, elem *nw) {
    if (el) {
        if (el->l1 == old) {
            el->l1 = nw;
        } else {
            el->l2 = nw;
        }
    }
```

Dwustronne kolejki ze scalaniem i odwracaniem

Example

```
void push_front(lista &l, int x) {
    elem *el = (elem*)malloc(sizeof(elem));
    el->val = x;
    el->l1 = l.head;
    el->l2 = l.head->l2;
    link(l.head->l2, l.head, el);
    l.head->l2 = el;
}

void push_back(lista &l, int x)
    analogicznie ...
```

Dwustronne kolejki ze scalaniem i odwracaniem

Example

```
void remove_elem(elem *el) {
    link(el->l1, el, el->l2);
    link(el->l2, el, el->l1);
    free(el);
}

void pop_front(lista &l) {
    if (!isempty(l)) {
        remove_elem(l.head->l2);
    }
}

void pop_back(lista &l) {
    analogicznie ...
```

Dwustronne kolejki ze scalaniem i odwracaniem

Example

```
void reverse(lista &l) {
    elem *tmp = l.head;
    l.head = l.tail;
    l.tail = tmp;
}
```

Dwustronne kolejki ze scalaniem i odwracaniem

Example

```
void append(lista &l1, lista &l2) {
    elem *b = l1.tail;
    elem *f = l2.head;
    link(b->l2, b, f->l2);
    link(f->l2, f, b->l2);
    b->l2 = f;
    f->l2 = b;
    l1.tail = l2.tail;
    l2.head = f;
    l2.tail = b;
}
```

Drzewa

- Implementacja drzew binarnych.
- Wysokość.
- Obejścia: in-order, pre-order, post-order.
- Metoda „spaceru”.

Drzewa

Example

```
typedef struct node *bin_tree;
struct node {
    int val;
    bin_tree left, right;
};

int height(bin_tree t) {
    if (t == NULL) return 0;
    return max(height(t->left), height(t->right)) + 1;
}
```

Drzewa

Example

```
void _pre_order(bin_tree t, vector<int> &res) {
    if (t) {
        res.push_back(t->val);
        _pre_order(t->left, res);
        _pre_order(t->right, res);
    }
}

vector<int> pre_order(bin_tree t) {
    vector<int> res;
    _pre_order(t, res);
    return res;
}
```

Drzewa

Example

```
void _in_order(bin_tree t, vector<int> &res) {
    if (t) {
        _in_order(t->left, res);
        res.push_back(t->val);
        _in_order(t->right, res);
    }
}

vector<int> in_order(bin_tree t) {
    vector<int> res;
    _in_order(t, res);
    return res;
}
```

Drzewa BST

- Niezmiennik danych drzew BST.
- Wyszukiwanie, wstawianie, usuwanie.
- Koszt czasowy $\Theta(h)$.
- Zrównoważone drzewa BST ($h = \Theta(\log n)$) — ASD.

Drzewa

Example

```
bin_tree* find(bin_tree *p, int el) {
    while ((*p) != NULL && (*p)->val != el) {
        if (el < (*p)->val) {
            p = &((*p)->left);
        } else {
            p = &((*p)->right);
        }
    }
    return p;
}
```

Drzewa

Example

```
bool member(bin_tree t, int el) {
    return (*find(&t, el)) != NULL;
}
void insert_bt(bin_tree &t, int el) {
    bin_tree *p = find(&t, el);
    if ((*p) == NULL) {
        *p = (bin_tree)malloc(sizeof(node));
        (*p)->val = el;
        (*p)->left = (*p)->right = NULL;
    }
}
```

Drzewa

Example

```
void delete_bt(bin_tree &t, int el) {
    bin_tree *p = find(&t, el);
    bin_tree r = *p;
    if (r == NULL) return;
    if (r->left == NULL && r->right == NULL) {
        *p = NULL;
    } else if (r->left == NULL) {
        *p = r->right;
    } else if (r->right == NULL) {
        *p = r->left;
    } else {
    ...
```

Drzewa

Example

```
        bin_tree *q = &((*p)->left);
        while ((*q)->right != NULL) {
            q = &((*q)->right);
        }
        r = *q;
        *q = (*q)->left;
        (*p)->val = r->val;
    }
    free(r);
}
```

Drzewa Find-Union

Example

- Struktura danych do reprezentowania relacji równoważności oraz jej klas abstrakcji.
- Alternatywnie: graf nieskierowany i spójne składowe.
- Interfejs:
  - typ reprezentujący elementy i klasy,
  - konstruktor elementów i par,
  - sprawdzenie równoważności,
  - reprezentant i elementy klasy abstrakcji,
  - liczba klas abstrakcji.

Implementacja drzew Find-Union

Example

```
typedef struct fu *find_union;

find_union create_fu(int label);

int find(find_union p);

bool equivalent(find_union p, find_union q);

void union_fu(find_union p, find_union q);

int size (find_union p);

vector<int> elements(find_union p);
```

Drzewa Find-Union

Example

- `find_union` — elementy klas abstrakcji etykietowane liczbami.
  Różne (w sensie ==) wartości sklejamy w jedną klasę abstrakcji.
- Każde wywołanie `create_fu` tworzy nowy element z podaną etykietą.
  Nawet jeśli dwa razy wywołamy z tą samą etykietą.
  Etykiety powinny być unikalne.
- Elementy x i y są w tej samej klasie abstrakcji gdy `find(x) == find(y)`.
  (Chyba, że etykiety nie są unikalne.)

Implementacja drzew Find-Union

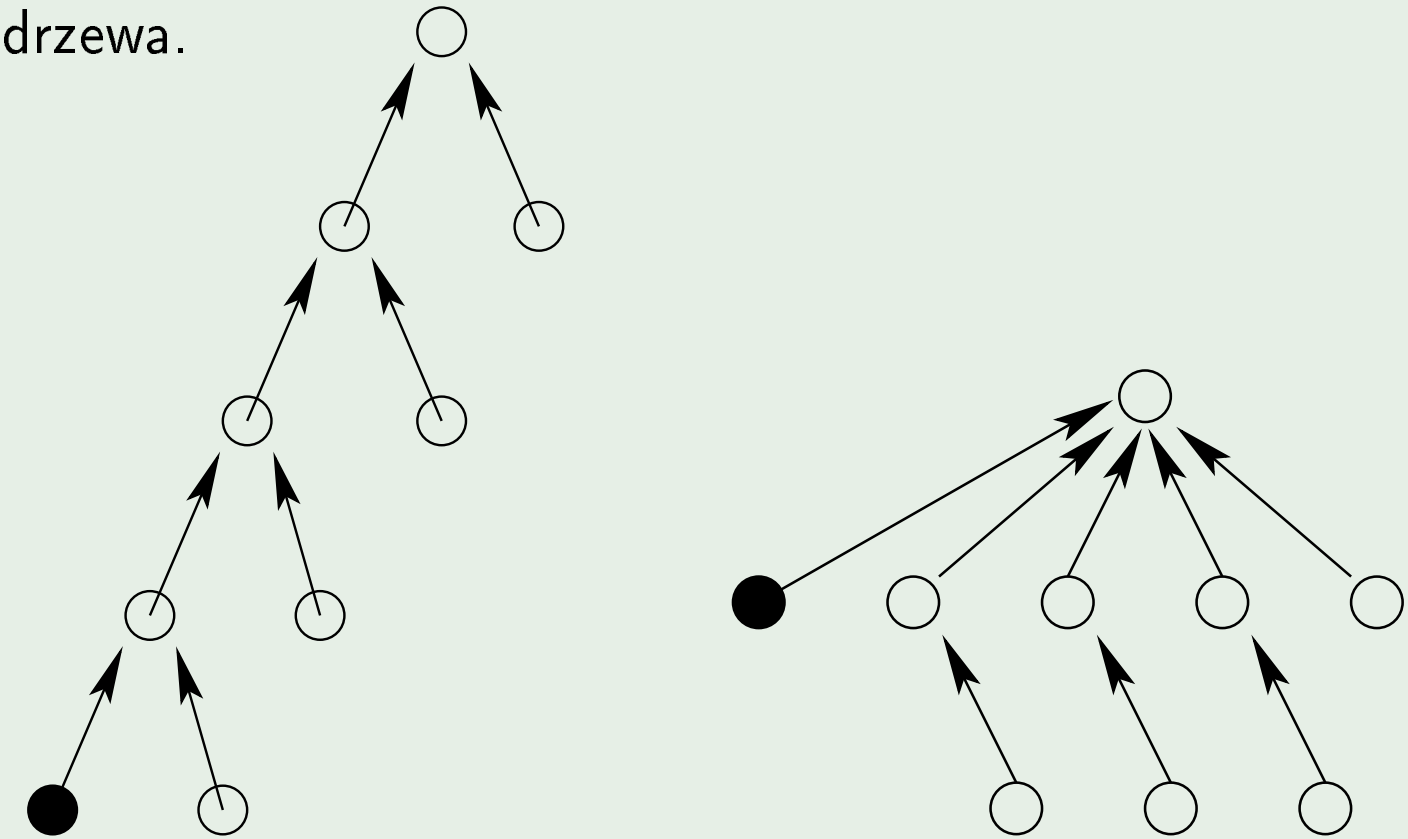
Example

- Klasy abstrakcji = drzewa.
  Węzły = elementy klas abstrakcji.
- Wskaźniki od węzła do ojca (w korzeniu `NULL`).
- Korzeń drzewa = reprezentant klasy.
- `create_fu` tworzy izolowany wierzchołek.
- `find` znajduje korzeń drzewa i zwraca jego etykietę.
- `union` znajduje korzenie dwóch drzew i podłącza jednego jako syna drugiego.
  Korzeń mniejszego drzewa podczepiamy pod korzeń większego.

Kompresja ścieżek

Example

- Gdy wyszukujemy reprezentanta klasy, we wszystkich odwiedzanych węzłach przestawiamy wskaźniki na korzeń drzewa.



Wybór nowego korzenia

Example

- Gdy scalamy dwa drzewa — który korzeń ma się stać korzeniem nowego drzewa?
- Generalnie, podłączamy mniejsze drzewo do większego.
- Każdemu wierzchołkowi przypisujemy *rangę*, liczbę całkowitą, górne ograniczenie na wysokość wierzchołka.
- Ranga pojedynczego wierzchołka wynosi 0.
- Kompresja ścieżek nie wpływa na rangi wierzchołków.
- Scalając podłączamy korzeń o mniejszej randze, do korzenia o większej randze.
  Jeżeli oba korzenie mają tę samą rangę, to wybieramy którykolwiek i zwiększamy jego rangę o 1.

Wyliczanie elementów klas

Example

- Jak wyliczyć elementy klas abstrakcji?
- Potrzebujemy wskaźników idących w dół drzewa.
- Dla każdej klasy abstrakcji utrzymujemy również drzewo „rozpinające" elementy klasy.
- Korzeniem tego drzewa jest reprezentant klasy abstrakcji.
- W każdym węźle drzewa pamiętamy ciąg (wektor) jego następników.
- Kształty drzew tworzonych przez wskaźniki idące „w górę" i „w dół" nie muszą się pokrywać.

Drzewa Find-Union

Example

```
typedef struct fu *find_union;

struct fu {
    int label;
    find_union up;
    int rank, size;
    vector<find_union> elems;
};
```

Implementacja drzew Find-Union

Example

```
find_union create_fu(int label) {
    find_union res =
        (find_union)malloc(sizeof(struct fu));
    *res = {
        .label = label,
        .up = NULL,
        .rank = 0,
        .size = 1,
        .elems{}
    };
    return res;
}
```

Implementacja drzew Find-Union

Example

```
find_union go_up(find_union p) {
    if (p->up == NULL) return p;
    return (p->up = go_up(p->up));
}

int find(find_union p) {
    return go_up(p)->label;
}

bool equivalent(find_union p, find_union q) {
    return go_up(p) == go_up(q);
}
```

Implementacja drzew Find-Union

Example

```
void union_fu(find_union p, find_union q) {
    find_union rp = go_up(p);
    find_union rq = go_up(q);
    if (rp == rq) return;
    if (rp->rank > rq->rank) {
        rq->up = rp;
        rp->size += rq->size;
        rp->elems.push_back(rq);
    } else {
        rp->up = rq;
        rq->size += rp->size;
        rq->elems.push_back(rp);
        if (rq->rank == rp->rank) rq->rank++;
    }
}
```

Implementacja drzew Find-Union

Example

```
int size (find_union p) {
    return go_up(p)->size;
}

void _traverse(find_union p, vector<int> &res) {
    res.push_back(p->label);
    for (find_union q : p->elems) {
        _traverse(q, res);
    }
}
vector<int> elements(find_union p) {
    vector<int> res;
    _traverse(go_up(p), res);
    return res;
}
```

Analiza kosztu

Lemma

Dla każdego węzła, który nie jest korzeniem, jego ranga jest mniejsza od rangi jego ojca.

Dowód.

- Dowód przebiega indukcyjnie po historii obliczeń.
- Operacja `union` podłączając jeden korzeń do drugiego zapewnia, że ojciec ma większą rangę niż nowy syn.
- Kompresja ścieżek zachowuje powyższą własność.



Analiza kosztu

Lemma

Dla dowolnego drzewa, którego korzeń ma rangę r,
liczba węzłów w drzewie wynosi przynajmniej 2^r.

Dowód (indukcyjny po historii obliczeń):

- Jednoelementowe drzewa mają rangę 0.
- Kompresja ścieżek nie zmienia liczby węzłów, ani rangi korzenia.
- Jeżeli scalamy drzewa, których korzenie mają różne rangi, to własność jest zachowana.
- Jeżeli scalamy drzewa, których korzenie mają rangę r,
  to każde z tych drzew ma przynajmniej 2^r węzłów.
  Po ich scaleniu powstaje drzewo, które zawiera przynajmniej 2^{r+1} elementów i którego korzeń ma rangę $r+1$.

□

Analiza kosztu

Fact

Dla dowolnej rangi r istnieje co najwyżej $\frac{n}{2^r}$ węzłów rangi $\geq r$.

Dowód.

- Ranga wierzchołków rośnie w wyniku operacji `union`.
- Usuńmy z obliczeń wszystkie operacje `union`, które tworzą wierzchołki o randze $> r$. Ich ranga pozostanie $= r$.
- Każdy wierzchołek rangi r jest korzeniem drzewa zawierającego przynajmniej 2^r węzłów.
- Ponieważ wszystkich wierzchołków jest n, więc wierzchołków rangi r (lub większej) może być co najwyżej $\frac{n}{2^r}$. □

Analiza kosztu

Wniosek

Rangi węzłów nie przekraczają $\lfloor \log_2 n \rfloor$.

Definition

Funkcja $\log_2^i x$ jest zdefiniowana standardowo.
Może ona być nieokreślona.
Funkcja $\log^*$ jest zdefiniowana następująco:

$$\log^* x = \min\{i \geq 0 : \log_2^i x \leq 1\}$$

Example

$$\log^* 4 = 2$$

$$\log^* 65000 = 4$$

Analiza kosztu

Theorem

Łączny koszt wykonania m operacji na drzewach find-union (bez operacji `elements`), spośród których n to operacje `make_set` wynosi $O(m \cdot \log^* n)$.

Definition

Zanim przystąpimy do dowodu, zdefiniujemy funkcję B, odwrotną do funkcji $\log^*$:

$$B(j) = \begin{cases} \underbrace{2^{\cdot^{\cdot^{\cdot^{2}}}}}_{j \text{ razy}} & \\ 2 & \text{jeśli } j \geq 1 \\ 1 & \text{jeśli } j = 0 \\ -1 & \text{jeśli } j = -1 \end{cases}$$

Analiza kosztu

Dowód

- Wierzchołki dzielimy na *bloki*, ze względu na $\log^*$ ich rang.
  Bloki numerujemy tymi wartościami.
  Blok numer j zawiera rangi od $B(j-1)+1$ do $B(j)$.
- Bloki mają numery od 0 do co najwyżej
  $\log^*(\log_2 n) = \log^* n - 1$.
  Wszystkich bloków jest $\leq \log^* n$.
- Pomijając koszt operacji go_up, koszt operacji wynosi $O(m)$.
  Możemy więc skupić się na koszcie operacji go_up.
- Pojedyncza operacja go_up przebiega ścieżkę od danego węzła do syna korzenia: $(x_0, x_1, \ldots, x_l)$.
  Rangi wierzchołków na ścieżce rosną.
  Wierzchołki na ścieżce należące do tego samego bloku występują po kolei.

Analiza kosztu

Dowód

- Koszt go_up przypisujemy wierzchołkom na ścieżce.
  Dzielimy go na koszt *związany ze ścieżką* i *związany z blokami*.
- Koszt związany z blokami przypisujemy ostatnim w blokach wierzchołkom na ścieżce, oraz synowi korzenia, x_{l-1}.
- Koszt związany z blokami jednej operacji go_up wynosi $O(1 + \log^* n)$.
  Koszt związany z blokami wszystkich operacji go_up wynosi $O(m \cdot \log^* n)$.
- Należy jeszcze pokazać, że łączny koszt związany ze ścieżkami jest również rzędu $O(m \cdot \log^* n)$.

Analiza kosztu

Dowód

- Jeżeli jakiemuś wierzchołkowi został raz przydzielony koszt za blok, to już więcej nie będzie mu przydzielony koszt za ścieżkę.
- Jeżeli wierzchołkowi został w danej operacji go_up przydzielony koszt za ścieżkę, to w wyniku tej operacji jego ojciec zmienił się na taki o wyższej randze.
- Jeżeli ten nowy ojciec należy do innego bloku, to wierzchołek już więcej nie będzie miał przypisanego kosztu za ścieżkę.
- Jeżeli dany wierzchołek należy do bloku nr j, to może on mieć przypisany koszt za ścieżkę co najwyżej $B(j) - B(j-1) - 1$ razy.

Analiza kosztu

Fact

Oznaczmy przez $N(j)$ liczbę wierzchołków w bloku nr j.
Mamy: $N(j) \leq \frac{n}{B(j)}$.

Dowód lematu.

Korzystając z faktu, że węzłów o randze $\geq r$ jest $\leq \frac{n}{2^r}$ mamy:

$$\begin{aligned} N(0) &\leq \frac{n}{2^0} = \frac{n}{B(0)} \\ N(j) &\leq \frac{n}{2^{B(j-1)+1}} = \frac{n}{2B(j)} < \frac{n}{B(j)} \end{aligned}$$

□

Analiza kosztu

Dowód.

Łączną liczbę opłat za ścieżki możemy oszacować:

$$\sum_{j=0}^{\log^* n-1} (N(j) \cdot (B(j) - B(j-1) - 1)) \leq$$

$$\leq \sum_{j=0}^{\log^* n-1} \left(\frac{n}{B(j)} \cdot (B(j) - B(j-1) - 1) \right) \leq$$

$$\leq \sum_{j=0}^{\log^* n-1} \left(\frac{n}{B(j)} \cdot B(j) \right) = \sum_{j=0}^{\log^* n-1} (n) \leq n \cdot \log^* n$$

Stąd, łączny koszt wszystkich operacji jest rzędu $O(m \log^* n)$. □

Analiza kosztu

- W praktyce czynnik $\log^* n$ można przyjąć za stały.
- Nie przekracza on 5 dla wszelkich praktycznych wielkości.
- $B(5) \geq 10^{19\,728}$
- Liczba atomów we Wszechświecie jest szacowana na 10^{80}.

Listy dwukierunkowe

Example

Listy dwukierunkowe

Example

Listy jednokierunkowe

Example